

```

import React, { useState, useCallback, useRef, useEffect } from "react";
import { Toaster as SonnerToaster, toast as sonnerToast } from "sonner";
import { X, Pin, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface ToastActionIconConfig {
  icon: LucideIcon;
  onClick: (id: string | number) => void;
  tooltipText?: string;
}

export interface CustomToastProps {
  id: string | number;
  message: string;
  description?: string;
  leadingIcon?: LucideIcon;
  isCloseable?: boolean;
  isPinnable?: boolean;
  actionIcons?: ToastActionIconConfig[];
}

interface ToastState {
  isPinned: boolean;
  isHovered: boolean;
}

const MAX_ACTION_ICONS = 4;

function CustomToastCard({
  id,
  message,
  description,
  leadingIcon,
  isCloseable = false,
  isPinnable = false,
  actionIcons = [],
  onDismiss,
}: CustomToastProps & { onDismiss: (id: string | number) => void }) {
  const [state, setState] = useState<ToastState>({
    isPinned: false,
    isHovered: false,
  });
  const toastRef = useRef<HTMLDivElement>(null);

```

```

const timeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);

const handleDismiss = useCallback(() => {
  onDismiss(id);
}, [id, onDismiss]);

const handlePinToggle = useCallback(() => {
  setState((prev) => ({ ...prev, isPinned: !prev.isPinned }));
}, []);

const handleActionClick = useCallback(
  (action: ToastActionIconConfig) => {
    action.onClick(id);
  },
  [id],
);

useEffect(() => {
  if (state.isPinned) {
    if (timeoutRef.current) {
      clearTimeout(timeoutRef.current);
      timeoutRef.current = null;
    }
  }
  return () => {
    if (timeoutRef.current) {
      clearTimeout(timeoutRef.current);
    }
  };
}, [state.isPinned]);

const handleMouseEnter = useCallback(() => {
  setState((prev) => ({ ...prev, isHovered: true }));
}, []);

const handleMouseLeave = useCallback(() => {
  setState((prev) => ({ ...prev, isHovered: false }));
}, []);

const limitedActionIcons = actionIcons.slice(0, MAX_ACTION_ICONS);

return (
  <div
    ref={toastRef}
    className={cn(
      "relative flex flex-col gap-2",
      "bg-card/80 backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-[var(--radius-surface)]",
      "shadow-[var(--shadow-md)]",
    )}
  >

```

```

    "text-text-main",
    "transition-all duration-200 ease-out",
    "min-w-[320px] max-w-[480px]",
    "p-4",
    state.isHovered &&
      "shadow-[var(--shadow-lg)]
border-[color-mix(in_oklch,var(--color-border)_80%,transparent)]",
  )}
  onMouseEnter={handleMouseEnter}
  onMouseLeave={handleMouseLeave}
  >
  <div className="flex items-start gap-3">
    {leadingIcon && (
      <div className="flex-shrink-0 mt-0.5 text-current opacity-80">
        {React.createElement(leadingIcon, {
          size: 20,
          strokeWidth: 2,
          "aria-hidden": "true",
        })}
      </div>
    )}
    <div className="flex-1 min-w-0">
      <p className="font-medium text-sm leading-tight text-foreground">
        {message}
      </p>
      {description && (
        <p className="mt-1 text-sm leading-relaxed text-muted-foreground">
          {description}
        </p>
      )}
    </div>
    <div className="flex items-center gap-1.5 flex-shrink-0">
      {isPinnable && (
        <button
          type="button"
          onClick={handlePinToggle}
          className={cn(
            "p-1.5 rounded-md transition-all duration-150 ease-out",
            "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
            "active:scale-95",
            "text-muted-foreground hover:text-foreground",
            state.isPinned &&
              "text-brand-primary
bg-[color-mix(in_oklch,var(--brand-primary)_15%,transparent)]",
          )}
          aria-label={state.isPinned ? "Unpin toast" : "Pin toast"}
          aria-pressed={state.isPinned}
        >
          <Pin size={14} strokeWidth={2.5} aria-hidden="true" />
        </button>
      )}
    </div>
  </div>

```

```

    })
    {isCloseable && (
      <button
        type="button"
        onClick={handleDismiss}
        className={cn(
          "p-1.5 rounded-md transition-all duration-150 ease-out",
          "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
          "active:scale-95",
          "text-muted-foreground hover:text-foreground",
        )}
        aria-label="Dismiss toast"
      >
        <X size={14} strokeWidth={2.5} aria-hidden="true" />
      </button>
    )}
  </div>
</div>

{limitedActionIcons.length > 0 && (
  <div className="flex items-center justify-end gap-1.5 pt-1 border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
    {limitedActionIcons.map((action, index) => (
      <button
        key={index}
        type="button"
        onClick={() => handleActionClick(action)}
        className={cn(
          "p-2 rounded-md transition-all duration-150 ease-out",
          "hover:bg-[color-mix(in_oklch,var(--color-muted)_50%,transparent)]",
          "active:scale-95",
          "text-muted-foreground hover:text-foreground",
          "flex items-center justify-center",
        )}
        aria-label={action.tooltipText || `Action ${index + 1}`}
      >
        <action.icon size={16} strokeWidth={2.5} aria-hidden="true" />
      </button>
    ))}
  </div>
)}
</div>
);
}

```

```

export function toast(props: CustomToastProps) {
  const { id, ...rest } = props;
  return sonnerToast.custom(
    () => (
      <CustomToastCard

```

```

        id={id}
        {...rest}
        onDismiss={({dismissId) => sonnerToast.dismiss(dismissId)}
      />
    ),
    {
      id,
      duration: Infinity,
    },
  );
}

export function dismiss(id?: string | number) {
  sonnerToast.dismiss(id);
}

export function dismissAll() {
  sonnerToast.dismiss();
}

interface CustomToasterProps {
  position?:
    | "top-left"
    | "top-right"
    | "bottom-left"
    | "bottom-right"
    | "top-center"
    | "bottom-center";
  className?: string;
}

export function CustomToaster({
  position = "top-right",
  className,
}: CustomToasterProps) {
  return (
    <SonnerToaster
      position={position}
      className={cn("toaster-group", className)}
      toastOptions={{
        className: cn(
          "group",
          "bg-card/80 backdrop-blur-[var(--backdrop-blur)]",
          "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-[var(--radius-surface)]",
          "shadow-[var(--shadow-md)]",
          "text-text-main",
          "transition-all duration-200 ease-out",
        ),
        style: {

```

```
      "--normal-bg": "var(--card)",
      "--normal-text": "var(--foreground)",
      "--normal-border":
        "color-mix(in oklch, var(--border) 60%, transparent)",
    } as React.CSSProperties,
  })
  closeButton={false}
  expand={false}
  richColors={false}
/>
);
}

export default CustomToaster;
```